TEAM GUIDE

Git Guide

Commit, push and merge, step by step.

Includes shortcut commands



THE BASICS

How your work moves

Working directory
The files you edit right now.

→

Staging area
The changes you picked for the next commit.

→

Local repo
Your commits, on your computer.

→

Remote
The shared repo your team uses.

```
git add → git commit → git push
```

THE BASICS

One-time setup

```
git config --global user.name "Your Name"
git config --global user.email "you@company.com"
git config --global init.defaultBranch main
git config --global pull.rebase false
git config --list
```

Do this once on each computer. The last command checks your settings.

THE BASICS

The daily workflow

1

Update main

`git pull`

2

Create a branch

`git switch -c feature/x`

3

Edit and commit

`git commit -m "..."`

4

Push your branch

`git push -u origin HEAD`

5

Open a merge request

On GitLab or GitHub

6

Merge after review

Then delete the branch

COMMIT

Save your work in small steps

```
git status
git diff
git add -p
git commit -m "Add CSV export"
git commit --amend --no-edit
```

Check what changed first.

Stage only what you mean to commit.

Commit small and often.

Never amend a commit that is already pushed.

COMMIT

Write clear messages

Good messages

```
Add CSV export to report page
Fix crash when track list is empty
Remove unused radar config
```

Weak messages

```
update
fix stuff
wip
```

One commit, one idea.

Start with a verb: Add, Fix, Remove. Keep the first line short.

COMMIT

Undo a mistake

Unstage a file	`git restore --staged file`
Discard edits in a file	`git restore file`
Undo last commit, keep changes	`git reset --soft HEAD~1`
Undo last commit, drop changes	`git reset --hard HEAD~1`
Undo a pushed commit	`git revert <hash>`

Warning: reset --hard deletes your work.

PUSH

Share your commits

```
git push -u origin HEAD
# first push of a new branch
git push
# every push after that
git pull
# when the push is rejected
git push --force-with-lease
# only on your own branch
```

Use -u on the first push of a new branch.

If the push is rejected, pull first, then push again.

Never force push to main.

MERGE

Bring a branch into main

Fast-forward

Main has no new commits. Git just moves the pointer.

Merge commit

Both branches have new commits. Git joins them with one commit.

```
git switch main && git pull
git merge feature/radar-filter
git push
```

MERGE

Fix a merge conflict

1 Run git status to see the files.

2 Open each file. Keep the final text.

3 Delete the marker lines.

4 Run git add, then git commit.

```
<<<<<<< HEAD
your version
=======
their version
>>>>>>> origin/main
```

Feeling lost? Run git merge --abort to start over.

MERGE OR REBASE?

Rebase your own branch. Merge shared branches.

Merge keeps the real history and is safe for everyone. Rebase makes a straight history, but it rewrites your commits.

```
git rebase main
```

SHORTCUT COMMANDS

Type less.

Make fewer mistakes.

Shell alias

```
gc "message"
```

Git alias

```
git cm "message"
```



SHORTCUT COMMANDS

Check, stage and commit

`gs`	`git status -sb`	Short status
`gd`	`git diff`	See your edits
`ga`	`git add`	Stage files
`gaa`	`git add .`	Stage everything
`gc "msg"`	`git commit -m "msg"`	Commit
`gca "msg"`	`git commit -am "msg"`	Stage tracked files and commit
`gcm`	`git commit --amend --no-edit`	Add to the last commit

SHORTCUT COMMANDS

Push, pull, branch and merge

`gp`	`git push`	Push
`gpu`	`git push -u origin HEAD`	Push a new branch
`gpf`	`git push --force-with-lease`	Safer force push
`gl`	`git pull`	Pull
`gsw`	`git switch`	Switch branch
`gswc`	`git switch -c`	Create a branch
`gm`	`git merge`	Merge a branch

SHORTCUT COMMANDS

Turn them on

```
# add to ~/.bashrc
alias gs='git status -sb'
alias gaa='git add .'
alias gc='git commit -m'
alias gp='git push'
alias gl='git pull'
```

Open ~/.bashrc (or ~/.zshrc) and paste your aliases.

Run **source ~/.bashrc** to load them.

If gs is Ghostscript on your machine, name it gss instead.

TEAM HABITS

Six rules for the team

Never commit directly to main.

Pull before you start and before you push.

Commit small, with clear messages.

Never force push to shared branches.

Keep secrets and build output out. Use .gitignore.

Delete branches after they are merged.

CHEAT SHEET

The whole workflow

Start, work and share

```
git switch main && git pull
git switch -c feature/my-task
git add -p
git commit -m "Clear message"
git push -u origin HEAD
```

Update and finish

```
git fetch && git merge origin/main
git switch main && git pull
git merge feature/my-task
git push
git branch -d feature/my-task
```